

Requirements reuse

Sylvia, the product manager at Tailspin Toys, was meeting with the development lead for their line of tablet apps for musicians. "Prasad, I just learned that Fabrikam, Inc., is going to release a larger version of their tablet, called a Substrate. Right now our guitar amplifier emulator runs on their smaller tablet, the ScratchPad. We need to come up with a version for the Substrate. We can do more with the larger screen. The Substrate will come with the new release of their operating system, which will run on both."

"Wow, this is great," said Prasad. "I'd like to be able to show more amp controls on the screen. We can make the controls bigger and easier to manipulate, too. We can reuse a lot of the core functionality from the ScratchPad emulator version. Unless Fabrikam changed the operating system APIs, we can reuse some of the code, too. We might want to drop some functionality in the ScratchPad version that our customers don't use. We can add the solid state/tube hybrid amp sounds from the web version, but we need to make some changes to suit the frequency response on the tablet. This should be fun!"

Reuse is an eternal goal for those seeking increased software productivity. People think most often in terms of code reuse, but many other software project components also have reuse potential. Reusing requirements can increase productivity and improve quality, as well as leading to greater consistency between related systems.

Reuse means taking advantage of work that has been done previously, whether on the same project or on an earlier project. Anytime you can avoid starting from scratch, you've got a running start on the project. The simplest way to reuse a requirement is to copy and paste it from an existing specification. The most sophisticated way is to reuse an entire functional component, from requirements through design, code, and tests. Numerous reuse options exist between these extremes.

Reuse is not free. It presents its own risks, both with respect to reusing existing items and to creating items with good reuse potential. It will likely take more time and effort to create high-quality reusable requirements than to write requirements you intend to use only on the current project. Despite the obvious merits, one study found that only about half of the organizations surveyed are actually practicing requirements reuse, primarily because of the poor quality of existing requirements (Chernak 2012). An organization that is serious about reuse needs to establish some infrastructure to make existing high-quality requirements knowledge accessible to future BAs and to foster a culture that values reuse.

This chapter describes several kinds of requirements reuse, identifies some classes of requirements information that have reuse potential in various contexts, and offers suggestions about how to perform requirements reuse. It presents some issues around making requirements reusable.

The chapter concludes with both barriers to effective reuse and success factors that can help your organization better take advantage of its existing body of requirements knowledge.

Why reuse requirements?

The benefits of effective requirements reuse include faster delivery, lower development costs, consistency both within and across applications, higher team productivity, fewer defects, and reduced rework. Reusing trusted requirements can save review time, accelerate the approval cycle, and speed up other project activities, such as testing. Reuse can improve your ability to estimate implementation effort if you have data available from implementing the same requirements on a previous project.

From the user's perspective, requirements reuse can improve functional consistency across related members of a product line or among a set of business applications. Consider the ability to format blocks of text by applying the same styling, spacing, and other properties in all members of a suite of related applications. Making this work in a uniform fashion involves reusing both functional and usability requirements. Such consistency can minimize the user's learning curve and frustration levels. It also saves time for stakeholders, who then will not need to specify similar requirements repeatedly.

Even if the implementation varies in different environments, the requirements might be the same. An airline's website might have a feature to let a passenger check in for a flight, pay for seat upgrades, and print boarding passes. The airline might also have self-service check-in kiosks at airports. The functionality for both check-in operations will be nearly identical, and hence reusable across the two products, even though the implementations and user experiences are dissimilar.

Dimensions of requirements reuse

We can imagine several types of requirements reuse. Sometimes a business analyst will recognize that a user-presented requirement resembles one from a previous project. Perhaps he can retrieve that existing requirement and adapt it for the new project. Such ad hoc reuse is most common with experienced BAs who have good memories and access to previous requirements collections. In other cases, a BA might use some existing requirements during elicitation to help users identify topics to consider for the new system. It's easier to modify something that exists than to create something new.

Table 18-1 describes three dimensions of requirements reuse: the extent of assets being reused, the extent to which an item must be modified for use in its new setting, and the mechanism being used to perform the reuse. When you're contemplating reusing requirements information, think about which option in each of these dimensions is most appropriate and practical for meeting your objectives.

TABLE 18-1 Three dimensions of requirements reuse

Dimension	Options
Extent of reuse	■ Individual requirement statement ■ Requirement plus its attributes ■ Requirement plus its attributes, context, and associated information such as data definitions, glossary definitions, acceptance tests, assumptions, constraints, and business rules ■ A set of related requirements ■ A set of requirements and their associated design elements ■ A set of requirements and their associated design, code, and test elements
Extent of modification	■ None ■ Associated requirement attributes (priority, rationale, origin, and so on) ■ Requirement statement itself ■ Related information (tests, design constraints, data definitions, and so on)
Reuse mechanism	■ Copy-and-paste from another specification ■ Copy from a library of reusable requirements ■ Refer to an original source

Extent of reuse

The first dimension has to do with the quantity of material that you are reusing. You might reuse just a single functional requirement. Or you might reuse such a statement along with any associated attributes, such as its rationale, origin, priority, and more if those are relevant to the target project. In some cases you can reuse not just the requirement but also associated artifacts: data definitions, acceptance tests, relevant business rules, constraints, assumptions, and so on. Often, a set of related requirements can be reused, such as all the functional requirements associated with a particular feature. Applications that run on similar platforms, such as different smartphone operating systems, could reuse requirements and design elements but perhaps not much code.

In the ideal scenario you can reuse a whole package of requirements, models, design components, code, and tests. That is, you reuse an entire chunk of implemented functionality essentially unchanged from a related product. This level of reuse can work when common operations are being employed across various projects on a common platform. Examples of such operations are error-handling strategies, internal data logging and reporting, communication protocol abstractions, and help systems. These functions must be developed for reuse with clear application programming interfaces (APIs) and all supporting documentation and test artifacts.



A reuse success tale

I once worked for a large retailer that was merging two online catalogs, one for consumers and one for corporations, into a single new system. The business objective was to reduce maintenance costs and make it easier to add new features that would appear in both catalogs. We developed the consumer catalog requirements first, based on the existing catalog functionality. For the corporate side, we started with those same consumer catalog requirements, then edited those that had to vary. Some new requirements were added for the new corporate catalog as well. The project delivered on schedule partly because of the time savings from reuse.

Extent of modification

The next dimension to consider is how much modification will be needed to make existing requirements reusable on the new project. In some cases, you'll be able to reuse a requirement unchanged. In the example given earlier about the airline's check-in kiosk, many of the functional requirements would be identical for the kiosk and for a website that offers passenger check-in. In other cases, you might reuse a requirement statement unchanged but have to modify some of its attributes, such as its priority or rationale as it applies to the new system. Often, you will start with an existing requirement but modify it to exactly suit the new purpose. Finally, whether or not you change the requirement, you might need to modify some designs and tests. An example is porting functionality from a PC to a tablet that has a touch screen rather than a mouse-and-keyboard interface.

Reuse mechanism

The most rudimentary form of reuse is simply a copy-and-paste of a piece of requirements information, either from another specification or from a library of reusable requirements. You don't retain a history of where the original information came from, and you can modify the copies you make. Copy-and-paste within a project increases the size of your specifications because you're duplicating information. If you find yourself populating a specification by doing a lot of copy-and-paste, a warning bell should ring. And just as when you copy code, copying and pasting requirements can introduce problems because of context issues, when the context isn't carried across with the paste operation.

In most cases, you're better off reusing existing content by referring to it instead of replicating it. This means that the original source of the information must be accessible to anyone who needs to view the requirement, and it must be persistent. If you're storing your requirements in a document and want the same requirement to appear in multiple places, you can use the cross-referencing feature of your word processor to link copies back to the master instance (Wiegers 2006). When the master instance is altered, the change is echoed everywhere you inserted a cross-reference link. This avoids the inconsistencies that can arise when one instance gets changed manually but others do not. However, it also runs the risk of all those requirements changing if someone else can alter the master instance.

Another copy-by-reference mechanism is to store not the actual requirement information but simply a pointer to it in your project documentation. Suppose you want to reuse descriptions of some user classes from other projects in your organization. First, collect such reusable information into a shared location. Possible forms for this collection include a word processing file, spreadsheet, HTML or XML file, database, and a specialized requirements tool. Give each object in that collection a unique identifier. To incorporate that information by reference, enter the identifier for each object you want to reuse in the appropriate section of your document. If technology allows, include a hyperlink directly to the reused object in the information collection. A reader who wants to view that user class description can simply follow the link to go to the master source. If you maintain that collection of reusable artifacts properly, those links and the destination information will always be current.

A much more effective way to reuse by reference is to store requirements in a requirements management tool, as described in Chapter 30, “Tools for requirements engineering.” Depending on the tool’s capabilities, you might be able to reuse a requirement that is already in the database without replicating it. Some such tools retain historical versions of individual requirements, which allows you to reuse a specific version of a requirement or set of related requirements. If someone modifies that requirement in the database, the older version that you are reusing still exists. You can then tailor your own version of that requirement to suit the needs of your project without disrupting other reusers.

Figure 18-1 illustrates this process. Project A creates the initial version of a particular requirement. Later on, Project B decides to reuse that same requirement, so the two projects share a common version. Then Project A modifies that requirement, thereby spawning version 2. However, version 1 still exists unchanged for use in Project B. If Project B needs to modify its copy later, it creates version 3, which does not affect any other project using any other version of that same requirement.

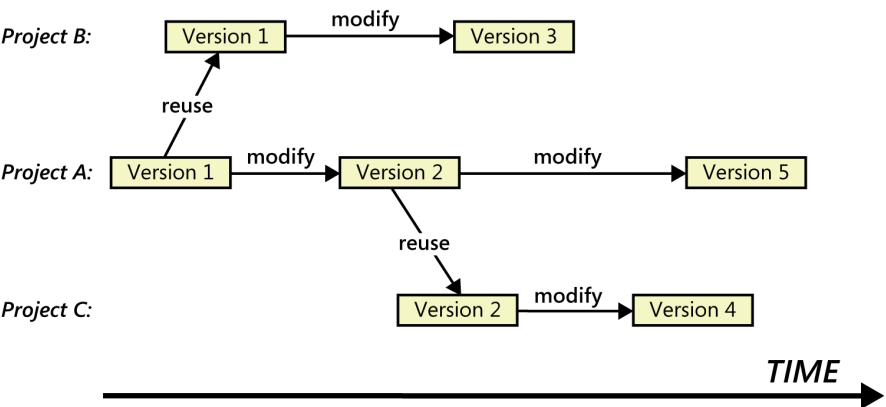


FIGURE 18-1 How a requirement can evolve through reuse in multiple projects.

Types of requirements information to reuse

Table 18-2 identifies some types of requirements-related assets that have good reuse potential for various scopes of applicability. Several of these assets appear in multiple scope categories. Some types of assets have very broad reusability, such as accessibility requirements (a subset of usability).

A set of related requirements in a specific functional area offers more reuse value than do single, isolated requirements. One example is around security (Firesmith 2004). There’s no reason for every project team in an organization to reinvent requirements for user logon and authentication, changing and resetting passwords, and so forth. If you can write a set of comprehensive, well-specified requirements for these common capabilities, they can be reused many times to save time and provide consistency across applications. You might be able to reuse sets of constraints within a specific operating environment or delivery platform. For instance, developers of smartphone apps need to be

aware of screen size, resolution, and user interaction constraints. Following are some other groupings of related requirements information to reuse in sets:

- Functionality plus associated exceptions and acceptance tests
- Data objects and their associated attributes and validations
- Compliance-related business rules, such as Sarbanes–Oxley, other regulatory constraints by industry, and organization policy-focused directives
- Symmetrical user functions such as undo/redo (if you reuse the requirements for an application’s undo function, also reuse the corresponding redo requirements)
- Related operations to perform on data objects, such as create, read, update, and delete

TABLE 18-2 Some types of reusable requirements information

Scope of reuse	Potentially reusable requirements assets
Within a product or application	User requirements, specific functional requirements within use cases, performance requirements, usability requirements, business rules
Across a product line	Business objectives, business rules, business process models, context diagrams, ecosystem maps, user requirements, core product features, stakeholder profiles, user class descriptions, user personas, usability requirements, security requirements, compliance requirements, certification requirements, data models and definitions, acceptance tests, glossary
Across an enterprise	Business rules, stakeholder profiles, user class descriptions, user personas, glossary, security requirements
Across a business domain	Business process models, product features, user requirements, user class descriptions, user personas, acceptance tests, glossary, data models and definitions, business rules, security requirements, compliance requirements
Within an operating environment or platform	Constraints, interfaces, infrastructures of functionality needed to support certain types of requirements (such as a report generator)

Common reuse scenarios

Whether you are creating a family of products, building applications across an organization, or even developing a product having a feature that appears in multiple contexts, there are opportunities for reuse. Let’s look at some scenarios where requirements reuse offers good potential.

Software product lines

Anytime you’re creating a set of products in a family—a software product line—those products will have a lot of functionality in common. Sometimes you’re producing variations of a base product for different customers or markets. Requirements you’ve incorporated into a specific variant for a particular customer might be folded into a common specification for the base product. Other product lines involve a family of related products that are based on a common architectural platform. For example, the vendor of a popular income tax preparation package offers a free version for online

use as well as basic, deluxe, premier, home and business, and business versions for use on personal computers. Analyze the features in the software product line to see which ones are:

- Common, appearing in all members of the product line.
- Optional, appearing in certain family members but not others.
- Variants, with different versions of the feature appearing in different family members (Gomaa 2004; Dehlinger and Lutz 2008).

The common features offer the greatest opportunities for reusing not just certain requirements, but also their downstream work products, including architectural components, design elements, code, and tests. This is the most powerful form of reuse, but we don't often detect the opportunity to take advantage of it. Reusing the common functionality is far better than reimplementing it each time, perhaps making it slightly different without good reason. Be aware of any constraints that the operating environment or hardware platform of certain products might impose that could limit reuse options. If the implementation must be different in certain product-line members, you're limited to reusing only requirements, not designs and code.

Reengineered and replacement systems

Reengineered and replacement systems always reuse some requirements from the original incarnation, even if those "requirements" were never written down. If you have to reverse-engineer requirements knowledge from an older system for possible reuse, you might need to move your thinking up to a higher level of abstraction to get away from specific implementation characteristics. Often, you can harvest business rules that were embedded in an old system and reuse them on future projects, updating them as necessary, as in the case of regulatory or compliance rules.

Trap Watch out for the temptation to reuse too much of an old system in the interest of saving time, thereby missing the opportunities offered by new platforms, architectures, and workflows.

Other likely reuse opportunities

Table 18-3 lists several other situations in which reusing requirements information is common. If you encounter any of these opportunities in your organization, contemplate whether it is worth accumulating the reusable artifacts into a shared repository and managing the information as an enterprise-level asset. If you previously worked on a project similar to the current one, consider whether you can use any artifacts from the earlier project again.

TABLE 18-3 Common opportunities where requirements reuse can be valuable

Reuse opportunity	Examples
Business processes	Often business processes are common across organizations and need to be commonly supported by software. Many institutions maintain a set of business process descriptions that are reused across IT projects.
Distributed deployments	Often the same system is deployed multiple times with slight variations. This is fairly typical for retail stores and warehouses. A common set of requirements is reused for each separate deployment.
Interfaces and integration	There is often a need to reuse requirements for interfaces and integration purposes. For example, in hospitals, most ancillary systems need interfaces to and from the admissions, discharge, and transfer system. This also applies to financial interfaces to an enterprise resource planning system.
Security	User authentication and security requirements are often the same across systems. For example, the systems might have a common requirement that all products must have a single sign-on using Active Directory for user authentication.
Common application features	Business applications often contain common functionality for which requirements—and perhaps even full implementations—can be reused. Possibilities include search operations, printing, file operations, user profiles, undo/redo, and text formatting.
Similar products for multiple platforms	The same core set of requirements is used even though there might be some detailed requirement and/or user interface design differences based on the platform. Examples include applications that run on both Mac and Windows or on both iOS and Android.
Standards, regulations, and legal compliance	Many organizations have developed a set of standards, often based on regulations, that are defined as a set of requirements. These are reused between projects. Examples are ADA Standards for Accessible Design and HIPAA privacy rules for healthcare companies.

Requirement patterns

Taking advantage of knowledge that makes the job of writing requirements easier can be regarded as reuse. That's the rationale behind requirement patterns: to package considerable knowledge about a particular type of requirement in a way that's convenient for a BA who wants to define such a requirement.

Pioneered by Stephen Withall (2007), a *requirement pattern* offers a systematic approach to specifying a particular type of requirement. A pattern defines a template with categories of information for each of the common types of requirements a project might encounter. Different types of requirement patterns will have their own sets of content categories. Populating the template will likely provide a more detailed specification of a requirement than if the BA simply wrote it in natural language. The structure and content of a requirement written according to a pattern facilitates reuse.

A requirement pattern contains several sections (Withall 2007):

- 1. Guidance** Basic details about the pattern, including related patterns, situations to which it is (and is not) applicable, and a discussion of how to approach writing a requirement of this type.
- 2. Content** A detailed explanation of the content that such a requirement ought to convey, item by item.

3. **Template** A requirement definition with placeholders wherever variable pieces of information need to go. This can be used as a fill-in-the-blanks starting point for writing a specific requirement of that type.
4. **Examples** One or more illustrative requirements of this type.
5. **Extra requirements** Additional requirements that can define certain aspects of the topic, or an explanation of how to write a set of detailed requirements that spell out what must be done to satisfy an original, high-level requirement.
6. **Considerations for development and testing** Factors for developers to keep in mind when implementing a requirement of the type specified by the pattern, and factors for testers to keep in mind when testing such requirements.

As an illustration, many software applications generate reports. Withall (2007) provides a pattern for specifying requirements that define reports. Withall's pattern includes a template that shows how to structure numerous report elements into a set of more detailed requirements that constitute a complete report specification. But the template is just one piece of the pattern. The pattern also contains an example of a reporting requirement, possible extra requirements to include, and considerable guidance about specifying, implementing, and testing such requirements.

You can create requirement patterns of your own that are ideally suited to your organization's style and projects. Following a pattern helps create consistency and will likely give you richer and more precise requirements. Simple templates like these remind you about important information that you might otherwise overlook. If you need to write a requirement on an unfamiliar topic, using a pattern is likely to be quicker than researching the topic yourself.

Tools to facilitate reuse

In an ideal world, your organization would store all of its software requirements in a requirements management tool with a complete set of traceability links. These links would tie each requirement back to a parent requirement or other origin, to other requirements it depends on, and to downstream development artifacts that are linked to it. Every historical version of each requirement would be available. This is the best way to enable effective reuse on a large scale across a whole application, product portfolio, or organization.

Few organizations have reached this level of sophistication, but storing your requirements in a tool will still enhance reuse in several ways (Akers 2008). Commercial requirements management tools provide various capabilities to facilitate reuse. Some even contain large libraries of requirements from certain domains available for ready reuse. When you're selecting a tool, include your expectations regarding how it will help you reuse requirements as part of the evaluation process. Chapter 30 describes typical capabilities of commercially available requirements management tools.

A tool might allow you to reuse a specific requirement by sharing it across multiple projects or baselines. If you do this, you need to think about what should happen if you change either the original requirement or its clones. Some tools let you lock the content so you can edit only

the original instance of the requirement. This ensures that any places where that requirement is reused are also updated at the time of the editing. Of course, if you start with a reused requirement and then *do* want to modify it for use in its new setting, you don't want to keep that lock in place. In that case, you would like to copy it using a mode that permits you to change the copied requirement.

Similarly, when you copy a requirement that has associated traceability relationships, you might or might not want to carry along everything that is linked to it. Sometimes, you might want to pull just the requirement, plus its children and requirements on which it depends, into a new project. This would be the case if the function is the same but the delivery platforms differ, as is the case with applications that run on a web browser, tablet, smartphone, and kiosk.

Trap If BAs can't find what they're looking for in the reuse repository, it doesn't matter how good the stored requirements are or how much time they might save: the BAs will write their own. Writing reusable requirements according to standard patterns provides a set of fields on which to search. Some people advocate adding meaningful keywords or requirement attributes to assist with searching.

Making requirements reusable

Just because a requirement exists doesn't mean it's reusable in its present form. It could be specific to a particular project. It might be written at too high a level because the BA could safely assume certain knowledge on the part of the development team or because some details were communicated only verbally. A requirement could be lacking information about how possible exceptions should be handled. You might have to tune up the original requirements to increase their value to future BAs.

Well-written requirements lend themselves to reuse. The steps you take to make requirements more reusable also increases their value to the project for which you originally write them; it simply makes them better requirements. Reusers need to know about dependencies each requirement has on others, as well as other requirements that go with it and that might also be reused, so that they can package sets of related requirements appropriately. Although *reuse* saves your team time and money, making something readily *reusable* is likely to cost time and money.

Reusable requirements must be written at the right level of abstraction and scope. Domain-specific requirements are written at a low level of abstraction. They are likely to be applicable only in their original domain (Shehata, Eberlein, and Hoover 2002). Generic requirements have broader applicability for reuse in a variety of systems. However, if you attempt to reuse requirements at too general a level, you won't save much effort because the BA will still have to elaborate the details. It's tricky to find the right balance between making reuse easier (with more abstract or generalized requirements) and making reuse pay off (with more detailed or specific requirements).

Figure 18-2 provides an example. Perhaps you're building an application that includes a user requirement to accept credit card payments. This user requirement would expand into a set of related

functional and nonfunctional requirements around handling credit card payments. Other applications also might need to take payments by credit card, so that's a potentially reusable set of requirements.

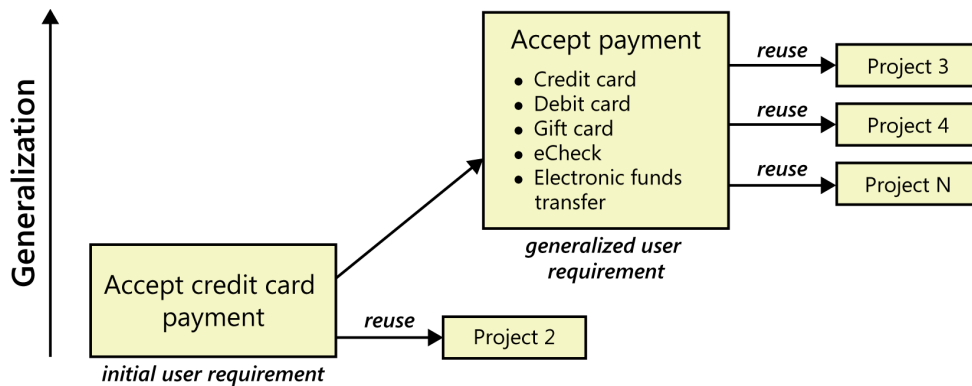


FIGURE 18-2 Generalized requirements offer greater reuse potential.

But suppose you could generalize that user requirement to encompass several payment mechanisms: credit card, debit card, gift card, eCheck, and electronic funds transfer. The resulting requirement offers greater reuse potential in a wider range of future projects. One project might need just credit card processing, whereas others require several of the payment processing methods. Generalizing an initial user requirement like this—from “accept credit card payment” to “accept payment”—could be valuable even on the current project. Even if the customer only asked to handle credit card payments initially, users might really like to accept multiple payment methods either now or in the future.



Choosing the right abstraction level for requirements can pay off during construction, as well. On one project that had exactly this need for multiple payment methods, generating clear requirements and rules for each case revealed both commonalities and distinctions. Independent from future reuse possibilities, building the higher-level abstractions contributed to easier design and construction.

That's the good news. The bad news is that it will take some effort to generalize the initially presented requirement. That's the investment you make in reusability, anticipating that you will recoup the investment—and more—through multiple, future reuse instances. It's up to you to decide whether to simply place today's requirements into a shared location for possible reuse or to invest effort to improve their reusability on future projects.



The “reusable requirements” explosion

A colleague offered a cautionary tale of how to reduce the potential value of reuse by writing excessively detailed requirements. A team tasked with writing requirements for a new project was obsessed with reuse. The BAs thought that if they documented all of the details for each requirement separately, then they could be reused. They ended up with more than 14,000 requirements! The repository contained entries that should have been just one requirement but had been structured as a parent with multiple child requirements, each giving a specific detail about the parent. Requirements this detailed were relevant only to that one application.

This volume of requirements also made the testing cycle much more difficult, leading to daily complaints from the testers. It was taking them much longer than expected to write test cases because they had to wade through such a vast quantity of requirements. The testers had to document the requirement ID in their test cases to ensure that test coverage of the requirements was achieved for traceability, but the number of traces on this many requirements became difficult to manage. In addition, the requirements underwent extensive change; they never did fully stabilize. All of these factors led to the project being deployed a year late, without producing the desired collection of reusable requirements.

Requirements reuse barriers and success factors

Requirements reuse sounds like a grand idea, but it isn't always practical or appropriate. This section describes some considerations to help your organization succeed with requirements reuse.

Reuse barriers

The first step to overcoming an obstacle is to recognize and understand it. Following are several barriers you might encounter when it comes to reusing requirements.

Missing or poor requirements A common barrier is that the requirements developed on previous projects weren't documented, so it's impossible to reuse them. And even if you find a relevant requirement, it might be badly written, incomplete, or a poor fit for your present circumstances. Even if they're documented, the original requirements for an old application might not have been kept current as the application evolved over time, rendering them obsolete.

NIH and NAH Two barriers to reuse are NIH and NAH syndromes. NIH means "not invented here." Some people are reluctant to reuse requirements from another organization or generic requirements found in a public collection. Requirements written elsewhere could be harder to understand: terminology could be different; the requirements might refer to documents that are unavailable; you might not be able to discern the context of the original requirements; and important background information could go unexplained. A BA might correctly decide that it takes less work to write new requirements than to understand and fix up the existing ones.

NAH, or "not applicable here," syndrome reveals itself when practitioners protest that a new process or approach does not apply to their project or organization. "We're different," they claim. The members might feel that their project is unique, so no existing requirements could possibly apply. Sometimes that's true, but often NIH and NAH indicate an inflexible attitude.

Writing style The BAs on previous projects might have used a wide variety of requirements representation techniques and conventions. It's best to adopt some standard notations for documenting requirements to facilitate reuse, such as using patterns. If requirements are written at a common level of granularity, it's easier for future BAs to search for candidate requirements at the right level of detail. Consistent terminology is also important. You might overlook a potentially

reusable requirement simply because some of the terminology involved is not the same as what your stakeholders are used to. Requirements written in natural language are notoriously prone to ambiguities, missing information, and hidden assumptions. These issues reduce their reuse potential.

Requirements that have embedded design constraints will offer little opportunity for reuse in a different environment. Think about the airport check-in kiosk described earlier. If user interface details about the kiosk were embedded in the requirements, you couldn't reuse those requirements for software having essentially the same functionality but running on a website.

Inconsistent organization It can be difficult to find requirements to reuse because authors organize their requirements in many different ways: by project, process flow, business unit, product feature, category, subsystem or component, and so forth.

Project type Requirements that are tightly coupled to specific implementation environments or platforms are less likely to generate reusable requirements or to benefit from an existing pool of requirements knowledge. Rapidly evolving domains don't yet have a pool of requirements information to reuse; requirements that are relevant today might be obsolete tomorrow.

Ownership Another barrier has to do with ownership (Somerville and Sawyer 1997). If you're developing a software product for a specific customer, its requirements are likely the proprietary intellectual property of the customer. You might not have the legal right to reuse any of those requirements in a different system you develop for your own company or for other customers.

Reuse success factors

An organization that is serious about reuse should create mechanisms to make it easy to share and take advantage of existing information. This means pulling information that has the potential for reuse out of a specific project so others can access and reuse it. Keep the following success tips in mind.

Repository You can't reuse something if you can't find it. An enabling tool for effective large-scale reuse, therefore, is a searchable repository in which to store requirements information. This repository could take several forms:

- A single network folder that contains previous requirements documents
- A collection of requirements stored in a requirements management tool that can be searched across projects
- A database that stores sets of requirements culled from projects for their reuse potential and enhanced with keywords to help future BAs know their origin, judge their suitability, and learn about their limitations

Consider giving someone the responsibility to manage the reusable requirements repository. This person would adapt existing requirements knowledge as necessary to represent and store the assets in a form suitable for efficient discovery, retrieval, and reuse. A scheme similar to that used to store and manage business rules as an enterprise asset could be adapted to handle reusable requirements.

Quality No one wants to reuse junk. Potential reusers need confidence in the quality of the information. And even if a requirement you are reusing isn't perfect, you should try to make it better when you reuse it. This way you iteratively improve a requirement over time, increasing its reuse potential for future projects.

Interactions Requirements often have logical links or dependencies on each other. Use traceability links in a tool to identify these dependencies so people know just what they're getting into when they select a requirement for reuse. Reused requirements must conform to existing business rules, constraints, standards, interfaces, and quality expectations.

Terminology Establishing common terminology and definitions across your projects will be helpful for reusability. Terminology variations won't prevent you from reusing requirements, but you'll have to deal with the inconsistencies and take steps to prevent misunderstandings. Glossaries and data dictionaries are good sources of reusable information. Rather than incorporating an entire glossary into every requirements specification, create links from key terms to their definitions in the shared glossary.

Organizational culture Management should encourage reuse from two perspectives: contributing high-quality components with real reuse potential, and effectively reusing existing artifacts. The individuals, project teams, and organizations that practice reuse most effectively are likely to enjoy the highest productivity. In a reuse culture, BAs look at the reusable requirements repository before creating their own requirements. They might start with a user story or other high-level requirement statement and see to what extent they can populate the details through reuse of existing information.

Your project requirements constitute valuable corporate information. To maximize the investment your teams make in requirements engineering, look for requirements knowledge that you can treat as an enterprise-level asset. The requirements you reuse do not have to be perfect to be valuable. Even if they just save you 20 percent of the work you might have otherwise spent writing new requirements, that's a big gain. A culture that encourages BAs to borrow first and create second, and that makes a little extra investment in making requirements reusable, can increase the productivity of both analysts and development teams and lead to higher-quality products.



Next steps

- Examine your current project to see if you can simplify requirements sets by reusing requirements knowledge from previous projects or other sources.
- Analyze your current project for requirements that are potentially reusable. Assess the scope of reuse from Table 18-2 for each requirement. Remember, you need to have a realistic chance of recouping the cost of extracting the reusable assets, packaging them, storing them, and making them accessible to others; otherwise, it's not a worthwhile investment.
- Think about what information you should store about your reusable requirements to make it easy for a future BA to search for them and judge whether they could be used on his project. Decide on a pragmatic repository in which to store requirements for reuse.